Caso práctico

Sistema de reservas para restaurante

Documento para resolver: requisitos del cliente (Parte A)
y resolución técnica de referencia (Parte B).

Stack de referencia: Laravel 12 · API REST · Sanctum · SQLite/MySQL
Código del proyecto en inglés · Comentarios explicativos en español.

Parte A — Caso del cliente

Contexto. Eres el equipo de desarrollo contratado por “La Mesa Verde”, un restaurante (con posibilidad de abrir más locales) que hoy toma reservas por teléfono y WhatsApp. Pierden turnos, se sobrebookean mesas y no tienen una visión clara de la carta ni del personal asignado a cada local. Quieren una **aplicación/API** que centralice la operación.

1. Quiénes usan el sistema

El dueño describe tres situaciones de uso cotidianas (sin entrar en detalles técnicos):

- **Clientes finales:** deben poder crear una cuenta, ver la carta de un local, consultar mesas libres para una fecha/hora y hacer o cancelar *su* reserva. No deberían ver ni modificar reservas de otras personas.
- **Personal del local:** necesitan gestionar el día a día: confirmar o reasignar reservas, registrar pedidos asociados a una reserva y consultar totales. No deberían poder borrar o reconfigurar toda la carta o los locales sin autorización.
- **Administración / gerencia:** deben poder dar de alta locales, mesas, platos, empleados y clientes; ver el panorama completo y corregir datos cuando haga falta.

En palabras del cliente: *“No cualquiera debería poder cambiar la carta o ver los datos de todos. Cada quien debe ver y hacer solo lo que le corresponde.”*

2. Qué deben poder gestionar

- **Locales (restaurantes):** nombre, dirección, teléfono y horario de apertura/cierre.
- **Mesas:** número, capacidad y ubicación (interior / terraza), pertenecientes a un local.
- **Carta (platos):** nombre, precio, descripción y tipo (entrada, fuerte, postre).
- **Empleados:** nombre y puesto laboral en el local (mesero, chef, gerente de piso).
- **Clientes:** nombre, correo y teléfono.
- **Reservas:** cliente, mesa, fecha, hora, número de personas y estado (pendiente, confirmada, cancelada).
- **Pedidos:** asociados a una reserva, con platos, cantidades y precio unitario; poder consultar el total de una reserva.

3. Reglas de negocio que el cliente exige

- No se puede reservar una mesa que ya esté ocupada (pendiente o confirmada) en la misma fecha y hora.
- El número de personas no puede superar la capacidad de la mesa.
- Debe existir una forma de listar mesas disponibles filtrando por fecha, hora, local y capacidad mínima.
- Debe existir un listado de platos más pedidos.
- Las respuestas de la API deben ser claras y consistentes (éxito, error, validación, listados paginados).
- Debe haber datos de ejemplo para probar el sistema sin cargar todo a mano.

4. Entregables esperados

- API REST documentada en el propio código (rutas, validaciones, recursos).
- Modelos con sus relaciones bien definidas.
- Validaciones de entrada (formularios/requests) y mensajes de error comprensibles.
- Estructura unificada de respuestas JSON.
- Datos semilla (seeders) y factories para pruebas.

- Control de acceso: solo usuarios autenticados realizan operaciones sensibles; cada tipo de usuario hace solo lo permitido.
- Pruebas automatizadas de los flujos críticos (registro/login, reserva, denegación de acceso).

5. Objetivos de aprendizaje (para el estudiante)

Al resolver este caso practicarás: modelos Eloquent y relaciones; factories y seeders; Form Requests y validaciones de negocio; API Resources / respuestas unificadas; autenticación por tokens; autorización (quién puede hacer qué); middleware; y policies.

Importante: en esta Parte A el cliente *no* menciona términos como “roles”, “permisos”, “middleware” o “policies”. Es tu trabajo interpretar el requisito “cada quien debe ver y hacer solo lo que le corresponde” e implementarlo en la Parte B / en el código.

Parte B — Resolución técnica

Esta sección interpreta el caso y describe la solución implementada en el repositorio **restaurant-reservas**. El código (tablas, rutas, campos, mensajes de validación) está en **inglés**; los comentarios de apoyo pueden estar en español.

1. Interpretación del requisito de acceso

El pedido del cliente se traduce en: autenticación con **Laravel Sanctum** (Bearer token); modelos propios **Role** y **Permission** (N:N) con el usuario; middleware `role` y `permission`; y una **ReservationPolicy** para que un customer solo gestione sus reservas. El campo `employees.position` (waiter/chef/manager) es el **puesto laboral**, distinto del rol de autenticación del sistema.

Roles y permisos sembrados

Rol	Permisos principales
admin	Todos (bypass Gate + todos los permisos)
manager	tables, dishes, employees, customers, reservations.manage, orders.manage
staff	reservations.manage, orders.manage, customers.manage
customer	reservations.own

Usuarios demo (password: `password`)

Email	Rol
admin@demo.test	admin
manager@demo.test	manager
staff@demo.test	staff
customer@demo.test	customer

2. Modelo de datos (inglés)

Tabla	Relaciones clave
restaurants	hasMany tables, dishes, employees
tables	belongsTo restaurant; hasMany reservations
customers	belongsTo user; hasMany reservations
dishes	belongsTo restaurant; belongsToMany orders
employees	belongsTo restaurant, user
reservations	belongsTo customer, table; hasOne order
orders	belongsTo reservation; belongsToMany dishes (order_dish)
roles / permissions	N:N con users y entre sí

3. Estructura de carpetas relevante

```
app/Models/                 # Restaurant, Table, Customer, ...
app/Traits/HasRoles.php     # assignRole, hasRole, hasPermission
app/Http/Controllers/Api/   # Auth + recursos CRUD
```

```
app/Http/Requests/          # Validaciones + authorize()
app/Http/Resources/         # Transformación JSON
app/Http/Responses/         # ApiResponse unificado
app/Http/Middleware/        # role / permission
app/Policies/               # ReservationPolicy
database/factories|seeders/ # Datos de práctica
routes/api.php              # Rutas públicas + auth:sanctum
```

4. Respuesta API unificada

Todas las respuestas pasan por `App\Http\Responses\ApiResponse`:

```
{
  "success": true,
  "message": "Success",
  "data": { ... },
  "meta": { "current_page": 1, "last_page": 3, "per_page": 15, "total": 40 }
}

# Error de validación
{
  "success": false,
  "message": "Validation failed",
  "errors": { "table_id": ["This table is not available..."] }
}
```

5. Endpoints principales

Método	Ruta	Acceso
POST	/api/auth/register|login	Público
GET/POST	/api/auth/me|logout	auth:sanctum
GET	/api/restaurants, /api/dishes	Público (lectura)
GET	/api/tables/available	Público
CRUD	/api/restaurants, tables, ...	auth + permisos
CRUD	/api/reservations	auth + policy
CRUD	/api/orders	permission:orders.manage
GET	/api/reservations/{id}/total	auth + policy
GET	/api/dishes/most-ordered	Público

6. Cómo ejecutar

```
composer install
cp .env.example .env   # si aplica
php artisan key:generate
touch database/database.sqlite
php artisan migrate:fresh --seed
php artisan serve
php artisan test
```

Ejemplos curl

```
# Login
curl -s -X POST http://localhost:8000/api/auth/login \
  -H 'Accept: application/json' -H 'Content-Type: application/json' \
  -d '{"email":"customer@demo.test","password":"password"}'

# Crear reserva (usa el token recibido)
```

```
curl -s -X POST http://localhost:8000/api/reservations \
  -H 'Authorization: Bearer TOKEN' -H 'Accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"table_id":1,"date":"2026-12-01","time":"20:00","party_size":2}'
```

7. Checklist de resolución

- Migraciones y modelos en inglés con relaciones Eloquent.
- Factories + seeders (incluye roles, permisos y usuarios demo).
- Form Requests con rules + withValidator (capacidad / disponibilidad).
- ApiResponse + Resources.
- Sanctum + HasRoles + middleware + ReservationPolicy.
- Tests Feature: auth, permisos, reservas y mesas disponibles.